You can satisfy these requirements by implementing *application logging*, which generates a record of events that take place within the running application. You can then refer to these log records to identify what happened, when it happened, and where it happened.

12.3 Logging

Logging is supported as a first-class citizen within ASP.NET Core. Several logging providers are enabled as part of the preconfigured defaults when the `WebApplication .CreateBuilder` method call is made in the `Program` class. A *logging provider* is responsible for the actual generation and output of log messages. Each one targets one or more different sources. The built-in providers enabled by default are the following:

- `Console`—Writes to the console
- `Debug`—Writes to the debug window in your IDE
- `EventSource`—Uses the operating system's event-tracing mechanism
- `EventLog`—Writes to the event log (Windows only)

In addition to these, you can enable your own logging provider. We will explore how to do that later in the chapter when you create a custom logger that gives you immediate notification that something has gone wrong in the application by outputting the log to email. You will learn about *structured*, or *semantic*, logging, which gives you rich data about the log and enables useful scenarios like querying and filtering of log messages. We will also look at integrating a third-party logging solution that includes support for ASP.NET Core and structured logging.